

Cascading Gradient Inversion via LT-Code Inspired Peeling in Federated Learning

Saeed Shariati and Mohsen Alambardar Meybodi

Abstract—Federated learning shares model updates rather than raw data, yet these updates can be inverted to reconstruct the clients’ training data. Analytic reconstruction attacks, which invert a gradient in closed form, degrade as the batch grows: prior single-round attacks recover only about half of a batch of size 100 even when the attacker fully controls the network parameters, and known upper bounds limit what any such method can recover. We establish a connection between gradient inversion and the theory of erasure-correcting codes, and use it to construct attacks that exceed these bounds. Our attacks recover batches exactly, together with every sample’s label, from a single FedSGD round, and certify each recovery without ground-truth data. On eight image and tabular benchmarks they outperform prior single-round attacks by a wide margin. Even a passive attacker who only observes an honestly trained network recovers 94–100% of ImageNet batches at sizes up to 128, more than prior single-round attacks achieve even with active manipulation of the model, and in the active setting more than 90% is recovered at batch sizes of several hundred. These results show that the privacy leakage of federated learning has been underestimated.

Index Terms—Gradient inversion, federated learning, FedSGD, analytic attack, Luby transform codes, privacy leakage

I. INTRODUCTION

Federated learning (FL) enables collaborative training of machine learning models without requiring participants to share their raw data [1], [2]. In each communication round, a central server distributes the current global model to a subset of clients; each client computes an update on its private local data and returns only the update, typically a gradient or model difference, to the server. Two widely studied algorithms are FedSGD, in which clients return the gradient of a local batch, and FedAvg, in which clients perform multiple local stochastic gradient steps before returning the net model update [1]. FL deployments are further divided into cross-silo and cross-device settings [2]: cross-silo clients are organizations that remain available throughout training, whereas cross-device clients are numerous mobile or IoT devices, only a fraction of which participate in each round and any of which may appear only once.

Although FL was designed with privacy in mind, numerous studies have shown that the shared updates can leak substantial information about clients’ training data. Such leakage has given rise to a variety of privacy attacks, which are commonly grouped into three categories: data reconstruction [3], [4], [5], [6], [7], [8], membership inference [9], [10], and property inference [11], [10]. These attacks may be either *active*,



Fig. 1: Original (top) and recovered (bottom) ImageNet samples from a single round of FedSGD.

where the server deliberately modifies weights, biases, or even the model architecture to facilitate reconstruction [12], [13], [14], or *passive*, where an honest-but-curious server attempts to recover information solely by inspecting the received updates [15], [16]. In response, several defenses have been proposed, most notably differential privacy (DP) [17] and secure aggregation (SA) [18], [19]. However, secure aggregation alone has been shown to be insufficient against certain privacy attacks, such as membership inference, and is therefore typically combined with differential privacy [20].

This paper focuses on *data reconstruction* attacks. Within this family one can further distinguish optimization-based [3], [4], [5], [21], [22], generative [23], [24], and analytic [6], [7], [25], [8], [26], [14] methods. Analytic attacks exploit explicit mathematical relationships between model gradients and the underlying training data. A prominent class of such attacks exploits closed-form relationships involving the first-layer gradients and input samples, typically for networks whose first layer is fully connected and followed by a ReLU activation. When a neuron is activated by only a single sample in the batch, that sample can be recovered exactly by a simple ratio of the corresponding weight and bias gradients [6]. Active analytic attacks are often easily detectable due to modifications of the model architecture [12], [27] or conspicuous changes in model parameters [7], [26], [14], while passive analytic attacks have so far struggled with large batch sizes [15].

Analytic attacks that rely on isolating individual samples, however, are fundamentally limited. Because they recover only isolated samples, the number of recoverable inputs is upper-bounded by the number of vertices of the convex hull formed by the batch [7]. This bound applies to isolation-based methods; it does not cover analytic attacks that recover batches through other mechanisms, such as the low-rank gradient structure exploited by SPEAR [15]. For data drawn from continuous distributions, the bound grows slowly with batch size and becomes particularly restrictive for low-dimensional (e.g., tabular) inputs. Consequently, single-round isolation-

Both authors are with the Department of Applied Mathematics and Computer Science, Faculty of Mathematics and Statistics, University of Isfahan, Isfahan 81746-73441, Iran (e-mail: saeed.shariati1380@gmail.com; m.alambardar@sci.ui.ac.ir).

Corresponding author: Mohsen Alambardar Meybodi.

based methods recover only a fraction of larger batches even under active model manipulation: with trap weights and 1000 neurons, Boenisch et al. [6] recover roughly half of a batch of 100 samples, and the fraction decreases as the batch grows. Multi-round variants [7], [8] recover full batches, but require repeated interaction with the client under carefully controlled per-round coefficients; this rules them out in the cross-device setting, where a client may be present in only a single round.

In this work we show that the limitation is not intrinsic. Once an isolated sample is recovered, its contribution can be subtracted from the observed gradient. The residual gradient is again of the same analytic form, but now computed over the remaining unrecovered samples. This subtraction can create new isolated samples, which can themselves be recovered and subtracted, producing an iterative cascade. The process is formally analogous to the peeling decoder of Luby Transform (LT) codes [28] (Section II-C).

To increase the length and success probability of the cascade, we further control the neuron degrees (Definition 1) of the first-layer neurons. Adapting the degree distribution of LT codes, we shape these degrees toward the robust soliton distribution. We introduce two novel constructions: Soliton-Free, which requires no auxiliary data and relies on a Gaussian approximation of pre-activations, and Soliton-Data, which uses a single auxiliary batch to set exact degree thresholds. Both constructions primarily adjust the first-layer biases to control neuron degrees.

Our attacks recover large batches exactly, together with their labels, from a single FedSGD communication round. The label of each recovered sample is obtained from the residual bias via a closed-form fit that also serves as a certificate of authenticity (Section V-C); no ground-truth data are required. The methods apply to any classification network whose first layer is a fully connected layer with ReLU activation, trained with cross-entropy loss (though they can be extended to a broader class of networks [6], [25]), and they succeed on both image and tabular data.

- We introduce a cascading recovery procedure for analytic gradient inversion that iteratively recovers and subtracts isolated samples, overcoming the upper bound that limits prior isolation-based attacks (Section III).
- We propose two parameter-manipulation techniques, Soliton-Free and Soliton-Data, that shape neuron activation degrees according to the robust soliton distribution, substantially increasing the fraction of recoverable samples. Soliton-Data recovers more than 90% of samples at batch sizes of several hundred on many data sets, and Soliton-Free recovers 98% of samples at $B = 300$ without auxiliary data (Table II).
- In the passive (honest-but-curious) setting, the attack achieves near-complete recovery (94–100%) at batch sizes up to 128 on image benchmarks and recovers a complete tabular batch at $B = 64$. For comparison, the passive state of the art SPEAR++ [16] reconstructs batches of at most about 65 samples at the same network width ($N = 1000$) and fails at $B = 150$, while prior single-round attacks failed to recover a batch size of 128 even in active settings [6].

- Recovered samples are exact and paired with their correct labels, with perfect label accuracy across eight data sets, and every recovery is certified without access to ground-truth data.

II. BACKGROUND

We write vectors in bold lowercase ($\mathbf{x}$) and matrices in bold uppercase ($\mathbf{X}$). Samples are indexed by i and neurons by j . Table I summarizes the notation used throughout the paper.

TABLE I: Notation

Symbol	Meaning
F	Input dimension
$\mathbf{x} \in \mathbb{R}^F$	A single input sample
$y \in \{1, \dots, K\}$	Label of $\mathbf{x}$; K = number of classes
$\mathbf{X} \in \mathbb{R}^{B \times F}$	Batch of B samples (one per row)
B	Batch size
N	Number of first-layer neurons
$\mathbf{W}_1 \in \mathbb{R}^{F \times N}, \mathbf{b}_1 \in \mathbb{R}^N$	First-layer weights and biases
$\mathbf{z} = \mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1$	First-layer pre-activations
$\mathbf{a} = \text{ReLU}(\mathbf{z})$	First-layer activations
$\mathbf{u} \in \mathbb{R}^K$	Logits; $\mathbf{p} = \text{softmax}(\mathbf{u})$
$\ell(\mathbf{x}, y)$	Per-sample cross-entropy loss
$\mathcal{L}$	Batch loss $\frac{1}{B} \sum_{i=1}^B \ell(\mathbf{x}_i, y_i)$
$c_{ij} = \partial \mathcal{L} / \partial z_{ij}$	Sensitivity of sample i at neuron j
c_s, δ	Robust-soliton parameters (Sec. II-C)
$\mathbf{G} \in \mathbb{R}^{F \times N}$	Weight gradient $\nabla_{\mathbf{W}_1} \mathcal{L}$
$\mathbf{h} \in \mathbb{R}^N$	Bias gradient $\nabla_{\mathbf{b}_1} \mathcal{L}$
$(\delta \mathbf{G}, \delta \mathbf{h})$	Recovered samples' first-layer gradient (Sec. V-D)
$\mathcal{F}$	Samples certified in the current iteration (Sec. V-D)
$\mathcal{K}$	Samples recovered so far (Sec. V-D)
$\mathbf{r}[j] \in \mathbb{R}^F$	Candidate vector at neuron j , $\mathbf{r}[j] = \mathbf{G}[:, j] / \mathbf{h}[j]$
A_j	Activation set $\{i : z_{ij} > 0\}$
$d_j = A_j $	Degree of neuron j
N_i	Set of neurons activated by sample i
$s \in (0, 1]$	Trap-weight scale
p_j	firing probability of neuron j ; $p_j = \frac{d_j}{B}$

A. Neural Networks for Classification

We consider classification networks that map an input $\mathbf{x} \in \mathbb{R}^F$ to logits $\mathbf{u} \in \mathbb{R}^K$, with class probabilities $\mathbf{p} = \text{softmax}(\mathbf{u})$. The only architectural assumption required by our attacks concerns the first layer: the input passes through a fully connected layer followed by a ReLU activation,

$$\mathbf{z} = \mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1 \in \mathbb{R}^N, \quad \mathbf{a} = \text{ReLU}(\mathbf{z}), \quad (1)$$

where $\mathbf{W}_1 \in \mathbb{R}^{F \times N}$ and $\mathbf{b}_1 \in \mathbb{R}^N$. All subsequent layers may be arbitrary. Training uses the cross-entropy loss. For a single sample $(\mathbf{x}, y)$,

$$\ell(\mathbf{x}, y) = -\log p_y, \quad \frac{\partial \ell}{\partial u_k} = p_k - \mathbb{I}[k = y]. \quad (2)$$

For a batch of B samples the loss is the average $\mathcal{L} = \frac{1}{B} \sum_{i=1}^B \ell(\mathbf{x}_i, y_i)$, and all gradients considered in this paper are taken with respect to this batch-averaged loss.

B. First-Layer Gradients

The attacks rely on the structure of the first-layer gradient. For a single sample $(\mathbf{x}, y)$, the first-layer pre-activation and activation of neuron j are

$$z_j = \mathbf{W}_1[:, j]^\top \mathbf{x} + \mathbf{b}_1[j], \quad a_j = \text{ReLU}(z_j),$$

and define the sensitivity of neuron j as $c_j = \partial \ell / \partial z_j$. Since

$$\frac{\partial a_j}{\partial z_j} = \mathbb{I}[z_j > 0] \implies c_j = \frac{\partial \ell}{\partial z_j} = \frac{\partial \ell}{\partial a_j} \mathbb{I}[z_j > 0],$$

$c_j = 0$ whenever the ReLU is inactive. Applying the chain rule once more yields the per-sample gradients

$$\nabla_{\mathbf{w}_1[:,j]} \ell = \frac{\partial \ell}{\partial z_j} \frac{\partial z_j}{\partial \mathbf{w}_1[:,j]} = c_j \mathbf{x}, \quad (3)$$

$$\nabla_{\mathbf{b}_1[j]} \ell = \frac{\partial \ell}{\partial z_j} \frac{\partial z_j}{\partial \mathbf{b}_1[j]} = c_j. \quad (4)$$

Consequently, when a neuron is activated by only one sample, the ratio of the corresponding weight and bias gradients recovers that sample exactly:

$$\frac{\nabla_{\mathbf{w}_1[:,j]} \mathcal{L}}{\nabla_{\mathbf{b}_1[j]} \mathcal{L}} = \mathbf{x}_i. \quad (5)$$

This identity is the basic recovery primitive used by prior analytic attacks [6] and by the iterative procedure developed in this work. For a batch, the first-layer gradients accumulate the contributions of all activating samples. Writing

$$\mathbf{G} = \nabla_{\mathbf{w}_1} \mathcal{L} \in \mathbb{R}^{F \times N}, \quad \mathbf{h} = \nabla_{\mathbf{b}_1} \mathcal{L} \in \mathbb{R}^N, \quad (6)$$

the j -th column satisfies

$$\mathbf{G}[:,j] = \sum_{i \in A_j} c_{ij} \mathbf{x}_i, \quad \mathbf{h}[j] = \sum_{i \in A_j} c_{ij}, \quad (7)$$

where $A_j = \{i : z_{ij} > 0\}$ is the activation set of neuron j and $c_{ij} = \partial \mathcal{L} / \partial z_{ij}$.

Definition 1 (Sensitivity, activation set, and degree). The *sensitivity* of sample i at neuron j is $c_{ij} = \partial \mathcal{L} / \partial z_{ij}$. The *activation set* of neuron j is $A_j = \{i : z_{ij} > 0\}$, and its *degree* is $d_j = |A_j|$.

Definition 2 (Singleton and isolated sample). A neuron of degree one is called a *singleton*. The unique sample that activates a singleton is said to be *isolated* at that neuron.

If sample i is isolated at neuron j , then (7) reduces to a single term and the ratio $\mathbf{G}[:,j] / \mathbf{h}[j]$ recovers $\mathbf{x}_i$ exactly.

C. Luby Transform Codes

The cascading recovery procedure introduced in Section V is analogous to the peeling decoder of Luby Transform (LT) codes [28]. In an LT code, a set of B message bits is encoded into a stream of check bits, each formed as the XOR of a randomly chosen subset of the message bits. Decoding proceeds by iteratively locating degree-1 check bits (those with only one message bit), recovering the associated message bit, and subtracting (XORing) its contribution from all neighboring check bits. This operation reduces the degrees of the remaining checks and frequently creates new degree-1 nodes, resulting in a cascade that recovers the entire message with high probability when the check-degree distribution is appropriately chosen.

In the present setting, the B input samples play the role of message bits, while the N first-layer neurons act as check nodes. A neuron of degree one isolates a single sample, which

can be recovered exactly via the ratio of the corresponding weight-gradient and bias-gradient entries. Subtracting the recovered sample's contribution from the residual gradient is analogous to the peeling step and can produce new isolated neurons, thereby propagating the recovery cascade. This parallel is illustrated in Fig. 2.

LT Encoding.

Given B message bits $m_1, \dots, m_B \in \{0, 1\}$, each check bit is generated as follows:

- 1) Draw a degree d from a prescribed distribution $\rho(d)$ supported on $\{1, \dots, B\}$.
- 2) Select a subset $A_j \subset \{1, \dots, B\}$ of size d uniformly at random.
- 3) Form the check bit

$$t_j = \bigoplus_{i \in A_j} m_i.$$

The resulting bipartite graph has message nodes on the left and check nodes on the right, with an edge between message i and check j whenever $i \in A_j$. The degree of check node j is $d_j = |A_j|$.

In our gradient-inversion setting the same structure arises: each neuron j is activated by a subset A_j of the batch, and the observed gradient column $(\mathbf{G}[:,j], \mathbf{h}[j])$ encodes a linear combination of the corresponding samples. By controlling the first-layer parameters we can influence the distribution of the degrees d_j .

LT Decoding (Peeling).

Whenever a check node of degree one exists, its check bit equals the unique neighboring message bit. The decoder therefore proceeds as follows:

- 1) Locate a check node j with $d_j = 1$ and let m_i be its sole neighbor.
- 2) Recover $m_i \leftarrow t_j$.
- 3) For every remaining check k adjacent to m_i , update

$$t_k \leftarrow t_k \oplus m_i$$

and decrement d_k by one.

- 4) Remove the recovered message node and all of its incident edges.
- 5) Repeat until either all message bits are recovered or no degree-1 checks remain.

The success of this process is governed by the degree distribution of the check nodes. When the degrees follow the robust soliton distribution, recovery succeeds with high probability provided the number of checks satisfies

$$N \gtrsim B(1 + \varepsilon),$$

where the overhead $\varepsilon = O(\ln^2(B/\delta)/\sqrt{B})$ guarantees a failure probability of at most δ . This guarantee relies on uniformly random edge selection; the extent to which it transfers to our data-dependent setting is discussed in Section VI.

The Robust Soliton Distribution.

The ideal soliton distribution on $\{1, \dots, B\}$ is defined by

$$\rho_{\text{ideal}}(1) = \frac{1}{B},$$

$$\rho_{\text{ideal}}(i) = \frac{1}{i(i-1)}, \quad i = 2, \dots, B.$$

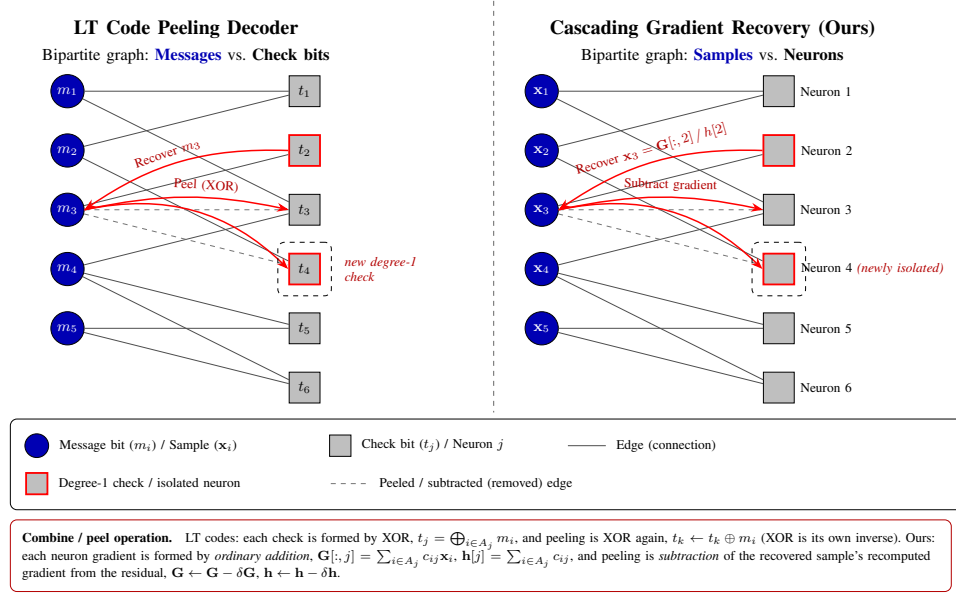


Fig. 2: Analogy between the LT-code peeling decoder (left) and the cascading gradient recovery process proposed in this work (right). In both cases, degree-1 nodes allow exact recovery of a connected left-hand node. Subtracting (peeling) the recovered node from the residual structure creates new degree-1 nodes, enabling an iterative cascade. In our setting, samples correspond to message bits and neurons correspond to check bits.

The robust soliton distribution is obtained by adding a carefully chosen perturbation $\tau(i)$ and renormalizing. Let

$$R = c_s \ln\left(\frac{B}{\delta}\right) \sqrt{B},$$

where $c_s > 0$ and $\delta \in (0, 1)$ are design parameters. The perturbation is given by

$$\begin{aligned} \tau(i) &= \frac{R}{iB}, \quad i = 1, \dots, \lfloor B/R \rfloor - 1, \\ \tau(\lfloor B/R \rfloor) &= \frac{R \ln(R/\delta)}{B}, \\ \tau(i) &= 0, \quad i > \lfloor B/R \rfloor. \end{aligned}$$

The resulting distribution places additional mass near degree B/R , which supplies a reservoir of higher-degree checks that sustain the peeling process after the initial wave of degree-1 recoveries.

In Sections VI-B and VI-C we deliberately shape the activation degrees of the first-layer neurons toward this robust soliton distribution, thereby maximizing recovery.

III. RELATED WORK AND POSITIONING

We position our attack against prior work in four areas: sparsity-based sample isolation, recovery bounds for isolation-based methods, exact batch reconstruction, and label recovery.

a) Sparsity-based analytic attacks.: The closest prior work is the attack of Boenisch et al. [6] (CaH). Their method recovers only isolated samples (singletons) and introduces mirrored trap weights to induce a sparse activation pattern whose degrees approximately follow a Poisson distribution with mean one. Our approach differs in two fundamental respects. First, we continue recovery beyond the initial set of isolated samples through an iterative subtraction process that

creates new singletons, forming a recovery cascade. Second, we deliberately shape the neuron degree distribution toward the robust soliton distribution, which Luby [28] showed to be near-optimal for peeling-style decoding, rather than relying on a Poisson distribution.

b) Recovery bounds for isolation-based methods.: Diana et al. [7] proved that any attack relying solely on isolated samples is upper-bounded by the number of vertices of the convex hull of the batch. A sample isolated by a neuron corresponds geometrically to a vertex that the hyperplane defined by that neuron's weights and bias separates from the remaining samples. For data drawn from continuous distributions the resulting bounds grow slowly with batch size:

- 1) $O(B^{(F-1)/(F+1)})$ for the unit ball,
- 2) $O(\log^{F-1} B)$ for the unit hypercube,
- 3) $O(\log^{(F-1)/2} B)$ for the standard Gaussian [29].

These bounds become particularly restrictive in low dimension, which led the authors to identify tabular data as a weak point of existing sparsity-based attacks. The cascading procedure developed in this paper is not subject to the same limitation: after outer samples are subtracted, interior points become vertices of the remaining set and can themselves be isolated and recovered. Consequently, our Soliton-Data method recovers HARUS batches in full at sizes up to 512 (Table II).

c) Exact batch reconstruction.: The sparsity-based attacks discussed above recover samples in closed form from a single gradient, but only while they are isolated. Diana et al. [7] recover full batches exactly, but this requires multiple communication rounds with carefully controlled per-sample gradient coefficients. Such control is incompatible with cross-device deployments, where a client may participate in only one

round. Their follow-up [8] reduces the number of rounds and certifies the isolation of each recovered sample. In the honest-but-curious setting, SPEAR [15], which its authors describe as the first algorithm to reconstruct whole batches with $B > 1$ exactly, exploits the low-rank structure of gradients together with ReLU-induced sparsity, recovering ImageNet-scale inputs for batch sizes up to approximately 25, and SPEAR++ [16] improves its scalability. Li et al. [30] reformulate exact input reconstruction as the Hidden Subset Sum Problem. Our attack recovers batches exactly from a single FedSGD round, in both the passive and the active setting, by successive isolation and subtraction rather than multi-round search or optimization, and each recovered sample carries its label and a certificate obtained from the same residual fit, without any additional queries.

d) *Label recovery.*: Analytic label recovery from gradients began with iDLG [4], which extracts the label of a single sample from the signs of the final-layer gradient. Subsequent batch methods (GradInversion [31], RLG [32], iLRG [33]) recover only aggregate label statistics of the batch and operate exclusively on the last layer. In contrast, we recover the label of each individual reconstructed sample from the first-layer bias residual via the logit Jacobian. The same procedure simultaneously serves as a certificate that the recovered vector is a genuine training sample, which is required for the subtraction step of the cascade.

IV. THREAT MODEL AND ARCHITECTURAL ASSUMPTIONS

A. Threat Model

We consider a standard federated learning setting in which a central server distributes the current global model to a set of clients. Each client computes an update on its private local data and returns the update to the server. We focus on single-round FedSGD, in which the client returns the gradient of the loss evaluated on a local batch of size B . The server therefore observes the first-layer gradient components $(\mathbf{G}, \mathbf{h})$ defined in Section II-B. Unlike FedAvg, the client does not perform multiple local SGD steps before returning an update.

The adversary is the server. We distinguish two threat models:

- **Passive (honest-but-curious).** The server follows the prescribed protocol exactly: it transmits an unmodified model and observes only the gradients (or model updates) returned by the clients. No parameter manipulation is performed.
- **Active (malicious).** The server is permitted to modify the weights and biases of the model before distribution, while still observing only the returned gradients. In this work we propose both a weight-manipulation method (the trap-weight construction of Section VI-A) and bias-manipulation methods; for the latter we introduce two variants, one where the server has access to an auxiliary batch of data (Soliton-Data, Section VI-C) and one where it does not (Soliton-Free, Section VI-B).

Our passive attacks are evaluated on a standard random initialization of the first-layer weights.

B. Architectural Assumptions

The proposed attacks rely on a single architectural property: the network begins with a fully-connected layer followed by a ReLU activation. Formally, for an input sample $\mathbf{x}_i \in \mathbb{R}^F$ we have

$$\begin{aligned} \mathbf{z}_i &= \mathbf{W}_1^\top \mathbf{x}_i + \mathbf{b}_1 \in \mathbb{R}^N, \\ \mathbf{a}_i &= \text{ReLU}(\mathbf{z}_i), \end{aligned} \quad (8)$$

where $\mathbf{W}_1 \in \mathbb{R}^{F \times N}$ and $\mathbf{b}_1 \in \mathbb{R}^N$. All subsequent layers may be arbitrary; the classification logits may appear any number of layers downstream.

The remainder of the network influences the attack only through the per-sample, per-neuron sensitivities $c_{ij} = \partial \mathcal{L} / \partial z_{ij}$. Once a candidate sample has been recovered, the server can compute these sensitivities by a standard forward and backward pass through its own copy of the model.

V. METHOD

We refer to the iterative recover-and-subtract procedure developed in this section as *peeling*, or equivalently the *cascade*; both terms denote the same process. Throughout, $(\mathbf{G}, \mathbf{h})$ denotes the current *residual* gradient and bias, i.e., what remains of the observed gradient after the contributions of all recovered samples have been subtracted; initially the residual is the observed gradient itself.

A. The Baseline: Singleton Extraction

If neuron j is a singleton in the current residual, i.e., $A_j = \{i\}$ among the unrecovered samples, then (7) contains a single term and the unknown sensitivity coefficient cancels in the ratio

$$\mathbf{r}[j] = \frac{\mathbf{G}[:, j]}{\mathbf{h}[j]} = \frac{c_{ij} \mathbf{x}_i}{c_{ij}} = \mathbf{x}_i. \quad (9)$$

If instead $|A_j| \geq 2$, the same ratio returns a linear combination of the activating samples:

$$\mathbf{r}[j] = \frac{\mathbf{G}[:, j]}{\mathbf{h}[j]} = \sum_{i \in A_j} \lambda_{ij} \mathbf{x}_i, \quad \lambda_{ij} = \frac{c_{ij}}{\sum_{i' \in A_j} c_{i'j}}, \quad (10)$$

whose coefficients sum to one but need not be positive, because the sensitivities c_{ij} may differ in sign. The resulting combination generally lies outside the data range and does not correspond to a valid sample. Consequently, recovery is upper-bounded by the number of isolated samples, which are the vertices of the convex hull formed by the batch (see Section III).

B. Label Recovery

Subtraction of a recovered sample requires its label y_i , which is not observed. The label is recovered from the residual bias. Assume the loss is cross-entropy over softmax logits $\mathbf{u}_i$ (located anywhere downstream), so that

$$\frac{\partial \mathcal{L}}{\partial u_{ik}} = \frac{1}{B} (p_{ik} - \mathbb{I}[k = y_i]).$$

Let

$$\mathbf{J}_i = \frac{\partial \mathbf{u}_i}{\partial \mathbf{z}_i} \in \mathbb{R}^{K \times N} \quad (11)$$

be the Jacobian of the logits with respect to the first-layer pre-activations. The classification layer may sit arbitrarily far downstream. The server evaluates $\mathbf{J}_i$ for a recovered sample by automatic differentiation on its own copy of the model.

A single forward pass of the recovered sample fixes the intermediate activations. Thereafter, the label-independent combination $\mathbf{J}_i^\top \mathbf{p}_i$ is a vector–Jacobian product, obtained with one backward pass (propagating the cotangent $\mathbf{p}_i$ from the logits). Any individual column $\mathbf{J}_i[:, j]$ is obtained via a forward-mode Jacobian–vector product along $\mathbf{e}_j$. For a two-layer network, $\mathbf{J}_i = \mathbf{W}_2^\top$ is constant and no additional passes are required.

For a neuron j that currently isolates sample i , the residual bias entry equals the sensitivity, $\mathbf{h}[j] = c_{ij}$. Applying the chain rule component-wise,

$$\mathbf{h}[j] = c_{ij} = \left(\mathbf{J}_i^\top \frac{\partial \mathcal{L}}{\partial \mathbf{u}_i} \right)_j = \frac{1}{B} \left[(\mathbf{J}_i^\top \mathbf{p}_i)_j - \mathbf{J}_i[y_i, j] \right], \quad (12)$$

where the first term is label-independent and the second is label-dependent. Every term except the unknown label y_i is known. Since (12) holds exactly for the true label, the label is recovered by trying each of the K candidate labels and keeping the one that minimizes this residual:

$$\hat{y}_i = \arg \min_{\ell \in \{1, \dots, K\}} \sum_{j \in S_i} \left(\frac{1}{B} \left[(\mathbf{J}_i^\top \mathbf{p}_i)_j - \mathbf{J}_i[\ell, j] \right] - \mathbf{h}[j] \right)^2, \quad (13)$$

i.e., the label for which this residual vanishes, where S_i is the set of neurons that currently isolate sample i (those whose candidate $\mathbf{r}[j]$ equals $\hat{\mathbf{x}}_i$). The fit uses only those neurons: the label-independent term $\mathbf{J}_i^\top \mathbf{p}_i$ is computed once per sample, after which each candidate label requires one table lookup per neuron $j \in S_i$. The computational cost is one forward pass, one backward pass, and $|S_i|$ Jacobian–vector products per sample (typically a small number). When $\mathbf{J}_i$ is constant the procedure reduces to pure arithmetic. No per-class backward passes and no auxiliary data are required.

Proposition 1 (Identifiability). *Two labels $\ell \neq \ell'$ are indistinguishable under (13) only if $\mathbf{J}_i[\ell, j] = \mathbf{J}_i[\ell', j]$ for every neuron $j \in S_i$. For weights drawn from any continuous distribution and $|S_i| \geq 1$ this event has probability zero. Because the fit uses exclusively the neurons that isolate sample i , duplicate labels elsewhere in the batch do not interfere.*

C. Certification: Distinguishing Genuine Candidates

The ratio $\mathbf{r}$ (Eq. (10)) produces one candidate per neuron with a nonzero residual bias entry. As shown by (10), some of these candidates are linear combinations rather than true samples. The attacker must distinguish the two without access to ground-truth data, because subtraction is destructive: subtracting an invalid combination injects error into the residual and corrupts every subsequent recovery.

The same consistency check that recovers the label simultaneously serves as a certificate of authenticity. For a candidate $\hat{\mathbf{x}}$, the server simulates a forward and backward pass (possible in closed form because it owns every network parameter)

and compares the predicted sensitivity against the observed residual bias on the neurons that produced this candidate:

$$\varepsilon(\hat{\mathbf{x}}, \ell) = \left\| \frac{1}{B} \left[(\mathbf{J}^\top \hat{\mathbf{p}})_{S_{\hat{\mathbf{x}}}} - \mathbf{J}[\ell, S_{\hat{\mathbf{x}}}] \right] - \mathbf{h}[S_{\hat{\mathbf{x}}}] \right\|, \quad (14)$$

where $S_{\hat{\mathbf{x}}} = \{j : \mathbf{r}[j] = \hat{\mathbf{x}}\}$ is the set of neurons whose candidate equals $\hat{\mathbf{x}}$, i.e., the duplicates of $\hat{\mathbf{x}}$ in $\mathbf{r}$. For a genuine sample these are exactly the neurons that currently isolate it, so $S_{\hat{\mathbf{x}}}$ coincides with the set S_i of (13) once $\hat{\mathbf{x}}$ is certified as sample i . A candidate that is a genuine sample satisfies (12) exactly on these neurons. In contrast, a linear combination $\sum_i \lambda_i \mathbf{x}_i$ has no single sample behind it: its simulated activation mask and sensitivities are those of the combination itself, not of any summand, and no label makes $\varepsilon(\hat{\mathbf{x}}, \ell)$ small.

Deduplication.: A sample is typically isolated by several neurons simultaneously, so the same vector appears multiple times among the candidates $\mathbf{r}[j]$. Before certification, the candidates are compared pairwise and those within a small distance tolerance are merged: one copy is kept and certified once, and the neuron indices of all its duplicates are pooled into its set $S_{\hat{\mathbf{x}}}$. Candidates duplicating an already recovered sample are likewise dropped, and their neurons are recorded as having handed over that sample. Deduplication therefore determines exactly the sets used by (13) and (14), and ensures each distinct sample is certified at most once per iteration.

D. Peeling

After forming the ratios, each candidate is certified and, if accepted, is assigned a label. The contribution of every certified sample is then subtracted from the observed batch gradient. This subtraction can create new degree-1 neurons and thereby enable further recoveries, producing a cascade.

In the implementation all samples certified within one iteration are subtracted jointly. Their combined contribution is the gradient that would have been returned on a batch consisting solely of those samples, while the loss remains averaged over the original batch size B :

$$(\delta \mathbf{G}, \delta \mathbf{h}) \leftarrow \nabla_{(\mathbf{w}_1, \mathbf{b}_1)} \frac{1}{B} \sum_{\mathbf{x} \in \mathcal{F}} \ell(\mathbf{x}, \hat{\mathbf{y}}_{\mathbf{x}}), \quad (15)$$

$$\mathbf{G} \leftarrow \mathbf{G} - \delta \mathbf{G}, \quad \mathbf{h} \leftarrow \mathbf{h} - \delta \mathbf{h}. \quad (16)$$

Here $\mathcal{F}$ denotes the set of samples certified in the current iteration; we call $(\delta \mathbf{G}, \delta \mathbf{h})$ the recovered samples' first-layer gradient. It is obtained with one batched backward pass. After subtraction, the residual retains the same analytic form, now taken only over the still-unrecovered samples:

$$\mathbf{G}[:, j] = \sum_{i \in A_j \setminus \mathcal{K}} c_{ij} \mathbf{x}_i, \quad \mathbf{h}[j] = \sum_{i \in A_j \setminus \mathcal{K}} c_{ij}, \quad (17)$$

where $\mathcal{K}$ is the set of samples recovered so far. The singleton identity (9) continues to apply unchanged. A neuron of original degree k yields its k -th sample as soon as the other $k - 1$ samples have been peeled away. Each successful subtraction can therefore expose new singletons, allowing the recovery process to cascade.

Algorithm 1 The Peeling Decoder

Require: First-layer gradient ($\nabla_{\mathbf{w}_1} \mathcal{L}$, $\nabla_{\mathbf{b}_1} \mathcal{L}$); server's own parameters; batch size B

- 1: $\mathbf{G} \leftarrow \nabla_{\mathbf{w}_1} \mathcal{L}$, $\mathbf{h} \leftarrow \nabla_{\mathbf{b}_1} \mathcal{L}$, $\mathcal{K} \leftarrow \emptyset$
- 2: **repeat**
- 3: $\mathbf{r}[j] \leftarrow \mathbf{G}[:, j] / \mathbf{h}[j]$ for every neuron j $\triangleright$ candidates, Eq. (9)
- 4: Merge duplicates: group the neurons j whose candidates $\mathbf{r}[j]$ coincide, keep one candidate per group with pooled neuron set $S_{\hat{\mathbf{x}}}$, and drop duplicates of samples in $\mathcal{K}$ $\triangleright$ deduplication
- 5: $\mathcal{F} \leftarrow$ distinct candidates certified by (14), each with label $\hat{y}$ from the fit (13) against the current $\mathbf{h}$
- 6: $(\delta \mathbf{G}, \delta \mathbf{h}) \leftarrow \nabla_{(\mathbf{w}_1, \mathbf{b}_1)} \frac{1}{B} \sum_{\mathbf{x} \in \mathcal{F}} \ell(\mathbf{x}, \hat{y}_{\mathbf{x}})$ $\triangleright$ one batched backward pass, Eq. (15)
- 7: $\mathbf{G} \leftarrow \mathbf{G} - \delta \mathbf{G}$, $\mathbf{h} \leftarrow \mathbf{h} - \delta \mathbf{h}$, $\mathcal{K} \leftarrow \mathcal{K} \cup \mathcal{F}$ $\triangleright$ peel the certified set
- 8: **until** no new sample is certified

E. The Loop

The complete iterative procedure is summarized in Algorithm 1.

Each neuron can yield at most one sample, because a neuron becomes a singleton at most once. The process terminates when no remaining neuron isolates a single unrecovered sample. At that point every neuron with a nonzero residual bias entry is activated by at least two unknown samples, the ratio identity (9) no longer applies, and the residual gradient is final. Complete recovery of the batch occurs when this terminal state is never reached before all B samples have been extracted.

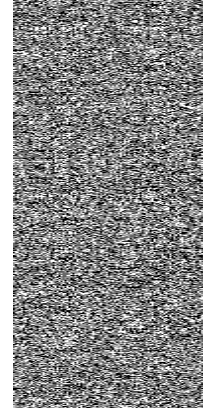
VI. PARAMETER MANIPULATION

To increase the length and success probability of the recovery cascade, we deliberately shape the activation degrees of the first-layer neurons. We first review trap-weight constructions that induce sparsity, and then present two methods that target the robust soliton distribution.

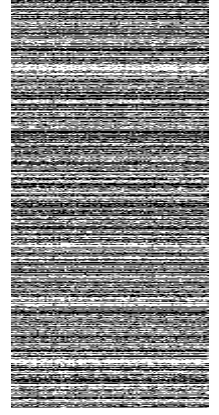
a) A caveat on the LT-code analogy.: Luby's bound assumes each check selects its edges uniformly at random; in our setting the edges are fixed by the data, since neuron j connects to sample i exactly when $\mathbf{W}_1[:, j]^\top \mathbf{x}_i + \mathbf{b}_1[j] > 0$. A sample inside the convex hull of the unrecovered samples can therefore never be the sole activator of a neuron until peeling shrinks the hull past it. Empirically it works well: Soliton-Data achieves near-complete recovery at batch sizes of several hundred (Table II).

A. Trap Weights

The server constructs each column of $\mathbf{W}_1$ so that activation is rare. The positive entries of a column are scaled by a constant $s \in (0, 1]$, making the column sum negative. Consequently, a generic input yields $z_{ij} < 0$ and is blocked by the ReLU. Only an input that is sufficiently correlated with the particular sign pattern of column j activates the neuron. The parameter s controls the strength of this asymmetry: as $s \rightarrow 1$ the asymmetry weakens and more neurons fire.



(a) Mirrored weights



(b) Independent weights

Fig. 3: Activation patterns (rows: neurons, columns: samples) at $s = 1$, $N = 1000$, $B = 100$. Left: mirrored construction. Right: independent construction. The mirrored pattern lacks the sparse rows that seed the cascade.

We compare two constructions; the entries of a column are correlated in the mirrored construction and independent in the other:

a) Mirrored construction.: Following Algorithm 1 of Boenisch et al. [6], a half-length vector of negative values is drawn and the positive half is defined as $-s$ times the same vector; both halves are then randomly permuted into place:

$$\mathbf{z}_- \sim -|\mathcal{N}(0, \sigma^2 \mathbf{I}_{F/2})|, \quad \mathbf{z}_+ = -s \mathbf{z}_-.$$

b) Independent construction.: Each entry retains its own magnitude and is independently assigned a positive sign with probability $1/2$, scaled by s :

$$\mathbf{W}_1[f, j] = \begin{cases} -|v_f| & \text{w.p. } \frac{1}{2}, \\ +s |v_f| & \text{w.p. } \frac{1}{2}, \end{cases} \quad \mathbf{v} \sim \mathcal{N}(0, \sigma^2 \mathbf{I}_F). \quad (18)$$

Both constructions use $\mathbf{b}_1 = \mathbf{0}$ and $\sigma = 0.5$, and both induce the same marginal distribution on the weight values. Their firing behavior, however, differs substantially.

c) Firing patterns.: Write each sample as $\mathbf{x}_i = \bar{\mathbf{x}} + \delta_i$ (batch mean plus deviation) and each neuron weight as $\mathbf{w}_j = m_j \bar{\mathbf{x}} + \mathbf{u}_j$ with $\mathbf{u}_j \perp \bar{\mathbf{x}}$. The pre-activation then decomposes as

$$\begin{aligned} z_{ij} &= \mathbf{w}_j^\top \mathbf{x}_i + \mathbf{b}_1[j] \\ &= m_j \|\bar{\mathbf{x}}\|^2 + \mathbf{w}_j^\top \delta_i + \mathbf{b}_1[j]. \end{aligned} \quad (19)$$

The resulting activation mask exhibits a stripe pattern: every row possesses its own baseline $m_j \|\bar{\mathbf{x}}\|^2$ determined by the alignment of the neuron with the mean, together with a per-sample deviation term. Neurons with large positive m_j fire on almost every sample (dense rows), while neurons with small or negative m_j fire only on atypical samples whose deviation pushes the pre-activation above zero. These sparse rows are what make singletons possible: if all neurons fired uniformly, a row of B samples would be a singleton with probability $B 2^{-B}$ (roughly 10^{-28} at $B = 100$) and the cascade could never begin. That the attack succeeds at all on a randomly initialized network (Table II, Passive rows) is direct evidence that firing is far from uniform, as Fig. 4 confirms.

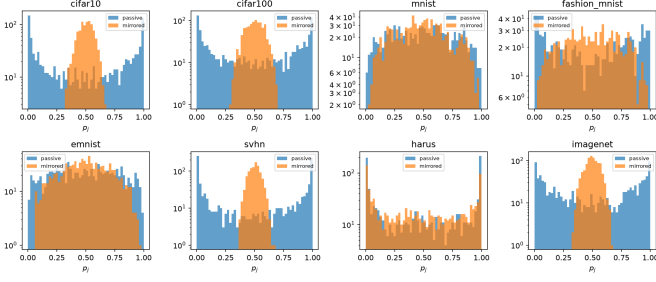


Fig. 4: Distribution of neuron firing probabilities p_j under passive (blue) and mirrored (orange) initialization, per data set. Mirroring concentrates p_j near $1/2$ only when the mean image is approximately constant; for MNIST, EMNIST, Fashion-MNIST, and HARUS the distribution stays dispersed.

d) Limitation of the mirrored construction.: At $s = 1$ the mirrored construction matches the marginal distribution of a standard Gaussian initialization, but it forces every column to sum to approximately zero. For data sets whose features concentrate about a common value a (Figure 7), this gives $\mathbf{w}_j^\top \bar{\mathbf{x}} \approx a \sum_f \mathbf{W}_1[f, j] \approx 0$: the baseline term in (19) vanishes, the stripe structure and its sparse bands disappear (Figure 3), and recovery collapses to nearly zero, as Boenisch et al. also observe. Figure 4 confirms this: mirroring drives the p_j toward uniform exactly for the data sets whose mean features concentrate around a single value, which by the argument above eliminates singletons.

e) Consequence for the passive setting.: The independent construction at $s = 1$ coincides exactly with a standard $\mathcal{N}(0, \sigma^2)$ initialization. Setting $s = 1$ therefore provides a meaningful no-tampering reference that corresponds to the honest-but-curious threat model.

B. Soliton-Free (SF)

Our goal is to make the neuron degrees $d_j = |A_j|$ follow a robust soliton distribution. Because the server has no access to client data, we adopt a Gaussian approximation based on mild independence assumptions. Features are assumed to be approximately uniform on the unit interval $[0, 1]$ and weakly dependent, so that $\mathbb{E}[x_f] \approx 1/2$, $\text{Var}[x_f] \approx 1/12$, and covariances are negligible. Under these conditions the pre-activation

$$z_j = \mathbf{w}_j^\top \mathbf{x} = \sum_{f=1}^F \mathbf{W}_1[f, j] x_f$$

is approximately Gaussian:

$$z_j \sim \mathcal{N}(\mu_j, \sigma_j^2),$$

with

$$\begin{aligned} \mu_j &= \mathbb{E}[z_j] = \frac{1}{2} \sum_{f=1}^F \mathbf{W}_1[f, j], \\ \sigma_j^2 &= \text{Var}[z_j] = \frac{1}{12} \sum_{f=1}^F \mathbf{W}_1[f, j]^2. \end{aligned}$$

Neuron j fires when $z_j + \mathbf{b}_1[j] > 0$, i.e., when $z_j > -\mathbf{b}_1[j]$. To ensure that the neuron activates on average for a prescribed degree d_j out of B samples, the threshold $-\mathbf{b}_1[j]$ is placed at the $(1 - d_j/B)$ -quantile of the Gaussian:

$$\Pr(z_j > -\mathbf{b}_1[j]) = \frac{d_j}{B}.$$

Solving with the inverse CDF Φ^{-1} of the standard normal distribution yields

$$\mathbf{b}_1[j] = -\mu_j - \sigma_j \cdot \Phi^{-1}\left(1 - \frac{d_j}{B}\right). \quad (20)$$

The degrees d_j themselves are drawn from the robust soliton distribution, thereby shaping the activation pattern toward the distribution that maximizes the success probability of peeling.

C. Soliton-Data (SD)

Soliton-Data likewise targets the robust soliton distribution, but exploits a single auxiliary batch to set the degrees exactly rather than through a Gaussian approximation. For each neuron j the server evaluates the pre-activations $\mathbf{w}_j^\top \mathbf{x}$ on the auxiliary samples and sorts them in decreasing order:

$$\mathbf{w}_j^\top \mathbf{x}_{(1)} \geq \mathbf{w}_j^\top \mathbf{x}_{(2)} \geq \dots \geq \mathbf{w}_j^\top \mathbf{x}_{(B_{\text{aux}})}.$$

Because neuron j fires on an input $\mathbf{x}$ precisely when $\mathbf{w}_j^\top \mathbf{x} > -\mathbf{b}_1[j]$, placing the threshold midway between the d_j -th and $(d_j + 1)$ -th largest values,

$$\mathbf{b}_1[j] = -\frac{\mathbf{w}_j^\top \mathbf{x}_{(d_j)} + \mathbf{w}_j^\top \mathbf{x}_{(d_j+1)}}{2}, \quad (21)$$

makes the activation set on the auxiliary batch exactly the d_j samples with the largest pre-activations. The auxiliary batch need not come from the clients: in our experiments it is drawn from the test split of each data set, which shares no samples with the clients' training data. When the auxiliary batch is statistically similar to the clients' data, the resulting degree distribution on the true batches closely matches the target robust soliton distribution.

VII. COMPUTATIONAL COST

Each iteration of the recovery procedure forms all candidate ratios $\mathbf{r}[j]$ in $O(FN)$ time. Duplicate candidates are then merged by pairwise comparison, so that each distinct sample is certified once and its duplicates contribute their neurons to its set $S_{\hat{\mathbf{x}}}$. Certification of each surviving candidate requires one forward pass and one backward pass (the vector-Jacobian product $\mathbf{J}_i^\top \mathbf{p}_i$) performed jointly across all candidates, together with one forward-mode Jacobian-vector product per singleton neuron. In practice, candidates lying outside the data range or possessing a near-zero bias entry are discarded before certification to improve efficiency.

The label fit (13) evaluates K candidate labels per candidate, and its residual is the certificate of Section V-C; after the $\mathbf{J}_i^\top \mathbf{p}_i$ term has been computed, it reduces to one table lookup per label ℓ and per neuron $j \in S_i$ (the neurons whose candidate $\mathbf{r}[j]$ equals $\hat{\mathbf{x}}_i$). Subtraction is performed jointly for all samples



Fig. 5: Top: samples that activate sparse neurons (neurons with degree less than 4). Bottom: typical batch samples that the cascade reaches late or never. The atypical samples are visibly off-distribution (darker, atypical ImageNet images). Results shown for $N = 1024$, $B = 512$, random initialization. m shows the mean pixel value of each image.

certified in the same iteration (Section V-D): a single backward pass evaluates the gradient of the combined loss

$$\frac{1}{B} \sum_{\mathbf{x} \in \mathcal{F}} \ell(\mathbf{x}, \hat{\mathbf{y}}_{\mathbf{x}}).$$

Consequently, the number of backward passes equals the number of iterations rather than the number of recovered samples. Memory consumption is dominated by the residual gradient matrix of size $O(FN)$.

VIII. EXPERIMENTS

a) Experimental setup.: We evaluate a two-layer multilayer perceptron with $N = 1000$ ReLU neurons and softmax cross-entropy loss on eight standard benchmark data sets: CIFAR-10 and CIFAR-100 [34], MNIST [35], EMNIST [36], Fashion-MNIST [37], SVHN [38], ImageNet [39], and HARUS [40]. Federated learning is simulated by a single FedSGD round on a batch of size B . All experiments are implemented in TensorFlow 2.21.0 and executed in 64-bit floating-point arithmetic on an NVIDIA GeForce RTX 4090 GPU paired with an Intel Core i9-14900K CPU. First-layer weights use $\sigma = 0.5$, second-layer weights use Xavier-uniform initialization [41], and all biases are zero except where the soliton constructions set $\mathbf{b}_1$. The code is publicly available.¹

A. Recovery Rate

Table II reports the average recovery rate across random seeds for eight data sets. The robust-soliton parameters (c_s, δ) are used only by the Soliton-Free and Soliton-Data variants; the trap-weight scale $s = 0.99$ is used only by the trap-weight baselines. For Soliton-Free and Soliton-Data we set $s = 1$. Each method is evaluated under both the independent and the mirrored weight constructions. Note that the trap-weight baselines are strengthened relative to the original CaH attack [6]: they run the full iterative peeling procedure with certification.

Soliton-Data consistently achieves the highest recovery rate, followed by Soliton-Free. In the passive setting the attack is weakest on MNIST and EMNIST, yet still recovers the majority of samples for batch sizes up to 128 on CIFAR-100, ImageNet, and several other data sets.

B. Which Samples Are Most at Risk

The entire cascade can be visualized on a single binary activation grid of size $N \times B$, where an entry (j, i) is marked if and only if $z_{ij} > 0$. Each column of the weight gradient is a linear combination of precisely the samples that activate the corresponding neuron (Eq. (7)). Consequently the singleton rows of the grid constitute the baseline attack, and every subtraction removes one column.

Figure 6 displays such a grid recorded from a real training batch of the testbed network ($N = 1024$, $B = 256$, $s = 1.00$) at successive iterations of the independent trap-weight attack. Rows are ordered by decreasing activation density and columns are ordered so that the most atypical samples appear on the right. Atypicality of sample i is quantified by

$$\text{atypicality}(i) = \sum_{j \in N_i} \frac{1}{p_j},$$

$$p_j = \frac{d_j}{B},$$

where N_i is the set of neurons activated by sample i and the p_j are computed once on the initial grid, before any column is deleted. In other words, a sample is atypical when it activates neurons that themselves fire infrequently.

Replaying the peeling process on the binary grid alone—locating singleton rows, deleting the corresponding column, and repeating—removes 17, 18, 13, 10 and 3 columns across five iterations and then stalls, recovering exactly 61 of the 256 samples (recovery rate 0.238). Every column traversed by the cascade is recoverable; every column never traversed remains unrecovered.

The deletions concentrate on the sparse bottom rows of the grid and overwhelmingly on the atypical columns stacked to the right. The samples recovered in the earliest iterations are therefore the atypical members of the batch. Visually they appear darker and off-distribution relative to typical images (Figure 5). Thus the cascade is seeded by the atypical samples.

IX. CONCLUSION AND FUTURE WORK

Prior isolation-based analytic gradient-inversion attacks recover only isolated samples and are therefore bounded by the number of vertices of the convex hull of the batch [7]. We showed that this limitation can be overcome by an iterative peeling process inspired by Luby Transform codes: once isolated samples are recovered and subtracted from the observed

¹<https://github.com/SaeedShariati/Gradient-Inversion-via-LT-Codes>

TABLE II: Average recovery rate across seeds (peeling attack, certificate-admitted) for $(c_s, \delta) = (0.05, 0.4)$, $s = 0.99$, $N = 1000$. Bold indicates recovery rate > 0.50 .

Dataset	Method	64	128	256	300	350	400	512	600	700	800	900	1024
CIFAR-10	SF-Ind	1.000	1.000	0.905	0.780	0.691	0.630	0.503	0.457	0.398	0.335	0.285	0.246
	SF-Mir	1.000	1.000	0.711	0.601	0.539	0.490	0.392	0.318	0.269	0.237	0.206	0.188
	SD-Ind	1.000	1.000	0.999	0.997	0.987	0.866	0.755	0.690	0.628	0.580	0.517	0.469
	SD-Mir	1.000	1.000	1.000	0.993	0.985	0.876	0.741	0.674	0.620	0.584	0.539	0.473
	TW-Ind	0.962	0.816	0.405	0.336	0.226	0.165	0.102	0.085	0.067	0.054	0.043	0.037
	TW-Mir	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
	Passive	0.994	0.666	0.215	0.183	0.130	0.098	0.070	0.055	0.040	0.038	0.033	0.027
	SF-Ind	1.000	1.000	0.984	0.875	0.729	0.677	0.500	0.457	0.408	0.350	0.271	0.226
	SF-Mir	1.000	1.000	0.700	0.600	0.520	0.460	0.376	0.325	0.288	0.250	0.225	0.195
	SD-Ind	1.000	1.000	0.999	0.986	0.915	0.812	0.691	0.611	0.545	0.487	0.465	0.411
CIFAR-100	SD-Mir	1.000	1.000	0.998	0.969	0.887	0.825	0.689	0.620	0.567	0.510	0.468	0.417
	TW-Ind	1.000	1.000	0.576	0.319	0.213	0.155	0.095	0.078	0.070	0.060	0.047	0.038
	TW-Mir	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
	Passive	1.000	1.000	0.247	0.205	0.149	0.114	0.082	0.055	0.051	0.041	0.029	0.019
	SF-Ind	1.000	1.000	0.998	0.567	0.341	0.266	0.193	0.156	0.128	0.107	0.096	0.080
	SF-Mir	1.000	1.000	1.000	1.000	1.000	0.983	0.518	0.317	0.251	0.222	0.187	0.148
	SD-Ind	1.000	1.000	1.000	0.999	0.998	0.997	0.981	0.968	0.913	0.712	0.471	0.322
	SD-Mir	1.000	1.000	1.000	1.000	0.993	0.986	0.966	0.956	0.911	0.674	0.451	0.346
	TW-Ind	0.503	0.030	0.005	0.003	0.002	0.001	0.000	0.000	0.000	0.000	0.000	0.000
	TW-Mir	0.031	0.003	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
EMNIST	Passive	0.672	0.019	0.005	0.003	0.002	0.001	0.000	0.000	0.000	0.000	0.000	0.000
	SF-Ind	1.000	1.000	1.000	0.992	0.749	0.417	0.307	0.234	0.181	0.140	0.125	0.109
	SF-Mir	1.000	1.000	1.000	0.987	0.961	0.764	0.486	0.290	0.224	0.173	0.152	0.126
	SD-Ind	1.000	1.000	1.000	1.000	0.976	0.991	0.925	0.792	0.695	0.582	0.501	0.437
	SD-Mir	1.000	1.000	1.000	1.000	1.000	0.973	0.870	0.710	0.666	0.601	0.546	0.466
	TW-Ind	1.000	0.997	0.061	0.026	0.014	0.012	0.008	0.005	0.002	0.001	0.001	0.001
	TW-Mir	0.469	0.017	0.001	0.001	0.001	0.001	0.000	0.000	0.000	0.000	0.000	0.000
	Passive	1.000	0.722	0.048	0.016	0.012	0.010	0.009	0.006	0.002	0.002	0.001	0.001
	SF-Ind	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.804	0.601	0.525	0.459	0.347
	SF-Mir	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.828	0.690	0.580	0.454	0.383
Fashion-MNIST	SD-Ind	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.804	0.601	0.525	0.459	0.347
	SD-Mir	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.828	0.690	0.580	0.454	0.383
	TW-Ind	1.000	0.997	0.061	0.026	0.014	0.012	0.008	0.005	0.002	0.001	0.001	0.001
	TW-Mir	0.469	0.017	0.001	0.001	0.001	0.001	0.000	0.000	0.000	0.000	0.000	0.000
	Passive	1.000	0.722	0.048	0.016	0.012	0.010	0.009	0.006	0.002	0.002	0.001	0.001
	SF-Ind	1.000	0.409	0.120	0.110	0.087	0.064	0.045	0.038	0.031	0.029	0.024	0.018
	SF-Mir	1.000	0.358	0.127	0.091	0.069	0.058	0.045	0.038	0.028	0.024	0.021	0.020
	SD-Ind	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.804	0.601	0.525	0.459	0.347
	SD-Mir	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.828	0.690	0.580	0.454	0.383
	TW-Ind	1.000	0.603	0.147	0.111	0.084	0.070	0.054	0.046	0.041	0.034	0.030	0.023
HARUS	TW-Mir	1.000	1.000	0.186	0.158	0.121	0.100	0.071	0.057	0.046	0.034	0.027	0.020
	Passive	1.000	0.475	0.115	0.091	0.074	0.060	0.046	0.035	0.030	0.024	0.022	0.020
	SF-Ind	1.000	1.000	0.993	0.980	0.876	0.817	0.669	0.589	0.471	0.396	0.332	0.287
	SF-Mir	1.000	1.000	0.960	0.828	0.723	0.617	0.521	0.462	0.401	0.363	0.325	0.296
	SD-Ind	1.000	1.000	0.995	0.973	0.925	0.896	0.744	0.657	0.629	0.538	0.495	0.448
	SD-Mir	1.000	1.000	0.995	0.992	0.970	0.841	0.764	0.651	0.615	0.576	0.523	0.425
	TW-Ind	1.000	1.000	0.838	0.718	0.608	0.474	0.413	0.311	0.245	0.199	0.146	0.114
	TW-Mir	0.612	0.617	0.616	0.613	0.614	0.604	0.593	0.498	0.341	0.016	0.004	0.001
	Passive	0.997	0.947	0.175	0.119	0.090	0.068	0.039	0.019	0.015	0.012	0.010	0.007
	SF-Ind	1.000	1.000	0.916	0.537	0.357	0.271	0.174	0.141	0.114	0.094	0.078	0.066
ImageNet	SF-Mir	1.000	1.000	0.999	0.985	0.975	0.963	0.473	0.293	0.236	0.192	0.147	0.125
	SD-Ind	1.000	1.000	1.000	1.000	1.000	1.000	0.978	0.956	0.785	0.637	0.529	0.393
	SD-Mir	1.000	1.000	1.000	1.000	1.000	0.995	0.975	0.913	0.799	0.695	0.608	0.481
	TW-Ind	0.362	0.022	0.004	0.003	0.002	0.002	0.001	0.001	0.000	0.000	0.000	0.000
	TW-Mir	0.031	0.006	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
	Passive	0.484	0.017	0.002	0.001	0.001	0.001	0.001	0.000	0.000	0.000	0.000	0.000
	SF-Ind	1.000	0.925	0.405	0.308	0.285	0.244	0.190	0.165	0.137	0.122	0.088	0.075
	SF-Mir	0.869	0.394	0.206	0.167	0.142	0.127	0.087	0.085	0.071	0.058	0.051	0.042
	SD-Ind	1.000	1.000	0.856	0.777	0.645	0.557	0.492	0.419	0.395	0.347	0.319	0.289
	SD-Mir	1.000	1.000	0.798	0.686	0.607	0.574	0.486	0.425	0.378	0.340	0.313	0.298
SVHN	TW-Ind	0.894	0.922	0.330	0.218	0.178	0.188	0.144	0.100	0.091	0.069	0.054	0.047
	TW-Mir	0.006	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
	Passive	0.959	0.881	0.268	0.177	0.147	0.113	0.086	0.079	0.063	0.057	0.047	0.040

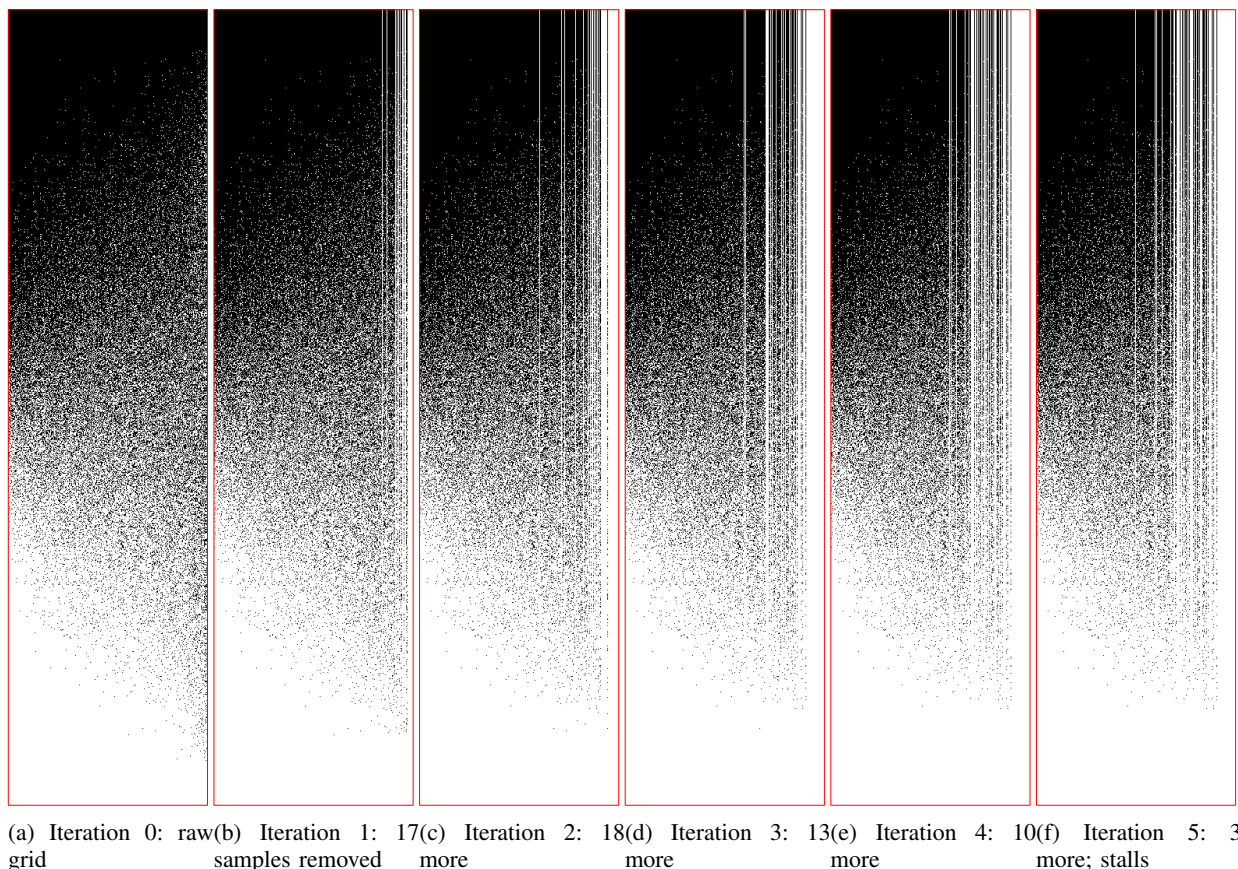


Fig. 6: Activation mask grid of a batch ($N = 1024$, $B = 256$, $s = 1.00$) at successive iterations of the independent trap-weight attack. A dot indicates that neuron j fires on sample i . Rows are ordered by decreasing firing density (dense rows at the top) and columns are ordered so that the most atypical samples appear on the right. The cascade removes 17, 18, 13, 10 and 3 samples across five iterations and then stalls, recovering $61/256 = 0.238$ of the batch—matching the recovery rate of the actual attack on this network. Peeling concentrates on the sparse bottom-right region where rare neurons intersect atypical samples.

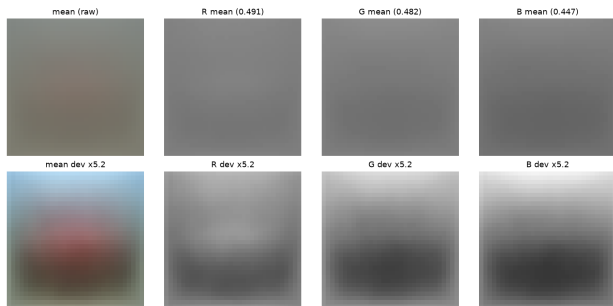


Fig. 7: Left: batch-mean image $\bar{x}$ (a blurred average CIFAR-10 picture), which constitutes the dominant term in Eq. (19). Right: per-channel mean deviations (amplified by $\times 5.2$). Neurons aligned with $\bar{x}$ fire on nearly all samples, while neurons aligned against it fire only on atypical samples; the latter sparse rows are the seeds of the recovery cascade.

gradient, previously non-isolated samples become isolated and can themselves be recovered. To maximize the cascade we further shape the first-layer activation degrees toward the robust soliton distribution via two constructions, Soliton-Free (no auxiliary data) and Soliton-Data (one auxiliary batch).

Experiments on eight image and tabular data sets demonstrate that the resulting attacks recover large batches exactly, together with their labels, from a single FedSGD round—near-complete recovery (94–100%) at batch sizes up to 128 in the passive setting and more than 90% at batch sizes of several hundred in the active setting—while certifying every recovery without ground-truth data.

Future work includes extending the cascade to FedAvg, deriving rigorous recovery guarantees under realistic data distributions, designing lightweight defenses that prevent the formation of degree-1 neurons, and investigating whether similar peeling attacks apply to networks with convolutional first layers.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2017.
- [2] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, R. G. L. D’Oliveira, S. E. Rouayheb, D. Evans, J. Gardner, Z. Garrett, A. Gascón, B. Ghazi, P. B. Gibbons, M. Gruteser, Z. Harchaoui, C. He, L. He, Z. Huo, B. Hutchinson, J. Hsu, M. Jaggi, T. Javidi, G. Joshi, M. Khodak, J. Konečný, A. Korolova, F. Koushanfar, S. Koyejo, T. Lepoint, Y. Liu, P. Mittal, M. Mohri, R. Nock, A. Özgür, R. Pagh, M. Raykova, H. Qi,

- D. Ramage, R. Raskar, D. Song, W. Song, S. U. Stich, Z. Sun, A. T. Suresh, F. Tramèr, P. Vepakomma, J. Wang, L. Xiong, Z. Xu, Q. Yang, F. X. Yu, H. Yu, and S. Zhao, “Advances and open problems in federated learning,” *Found. Trends Mach. Learn.*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [3] L. Zhu, Z. Liu, and S. Han, “Deep leakage from gradients,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019.
- [4] B. Zhao, K. R. Mopuri, and H. Bilen, “iDLG: Improved deep leakage from gradients,” *arXiv preprint arXiv:2001.02610*, 2020.
- [5] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, “Inverting gradients—how easy is it to break privacy in federated learning?” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.
- [6] F. Boenisch, A. Dziedzic, R. Schuster, A. S. Shamsabadi, I. Shumailov, and N. Papernot, “When the curious abandon honesty: Federated learning is not private,” in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroS&P)*, 2023, arXiv:2112.02918.
- [7] F. Diana, A. Nusser, C. Xu, and G. Neglia, “Cutting through privacy: A hyperplane-based data reconstruction attack in federated learning,” in *Proc. Conf. Uncertainty Artif. Intell. (UAI)*, 2025.
- [8] F. Diana, C. Xu, A. Nusser, and G. Neglia, “No more guessing: A verifiable gradient inversion attack in federated learning,” *arXiv preprint arXiv:2604.15063*, 2026.
- [9] M. Nasr, R. Shokri, and A. Houmansadr, “Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2019, pp. 739–753.
- [10] L. Melis, C. Song, E. D. Cristofaro, and V. Shmatikov, “Exploiting unintended feature leakage in collaborative learning,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2019, pp. 691–706.
- [11] K. Ganju, Q. Wang, W. Yang, C. A. Gunter, and N. Borisov, “Property inference attacks on fully connected neural networks using permutation invariant representations,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2018.
- [12] L. Fowl, J. Geiping, W. Czaja, M. Goldblum, and T. Goldstein, “Robbing the fed: Directly obtaining private data in federated learning with modified models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.
- [13] D. Pasquini, D. Francati, and G. Ateniese, “Eluding secure aggregation in federated learning via model inconsistency,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2022.
- [14] Y. Wen, J. Geiping, L. Fowl, M. Goldblum, and T. Goldstein, “Fishing for user data in large-batch federated learning via gradient magnification,” *arXiv preprint arXiv:2202.00580*, 2022.
- [15] D. I. Dimitrov, M. Baader, M. N. Müller, and M. Vechev, “SPEAR: Exact gradient inversion of batches in federated learning,” *arXiv preprint arXiv:2403.03945*, 2024.
- [16] A. Bakarsky, D. I. Dimitrov, M. Baader, and M. Vechev, “SPEAR++: Scaling gradient inversion via sparsely-used dictionary learning,” *arXiv preprint arXiv:2510.24200*, 2025.
- [17] C. Dwork, “Differential privacy,” in *Proc. Int. Colloq. Automata Lang. Program. (ICALP)*. Springer, 2006, pp. 1–12.
- [18] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, “Practical secure aggregation for privacy-preserving machine learning,” *Cryptology ePrint Archive*, Paper 2017/281, 2017. [Online]. Available: <https://eprint.iacr.org/2017/281>
- [19] E. Hosseini, S. Chen, and A. Khisti, “Secure aggregation in federated learning using multiparty homomorphic encryption,” *arXiv preprint arXiv:2503.00581*, 2025.
- [20] K.-H. Ngo, J. Östman, G. Durisi, and A. G. i Amat, “Secure aggregation is not private against membership inference attacks,” in *Proc. Eur. Conf. Mach. Learn. Princ. Pract. Knowl. Discov. Databases (ECML PKDD)*, 2024.
- [21] J. Qian, K. Wei, Y. Wu, J. Zhang, J. Chen, and H. Bao, “GI-SMN: Gradient inversion attack against federated learning without prior knowledge,” *arXiv preprint arXiv:2405.03516*, 2024.
- [22] J. Lu, X. S. Zhang, T. Zhao, X. He, and J. Cheng, “APRIL: Finding the achilles’ heel on privacy for vision transformers,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 10041–10050.
- [23] J. Jeon, J. Kim, K. Lee, S. Oh, and J. Ok, “Gradient inversion with generative image prior,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2021.
- [24] C. Zhang, X. Zhang, E. Sotthiwat, Y. Xu, P. Liu, L. Zhen, and Y. Liu, “Generative gradient inversion via over-parameterized networks in federated learning,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 5103–5112.
- [25] S. Zhang, J. Huang, Z. Zhang, P. Li, and C. Qi, “Compromise privacy in large-batch federated learning via model poisoning,” *Inf. Sci.*, vol. 647, p. 119421, 2023.
- [26] S. Shi, N. Wang, Y. Xiao, C. Zhang, Y. Shi, Y. T. Hou, and W. Lou, “Scale-MIA: A scalable model inversion attack against secure federated learning via latent space reconstruction,” *arXiv preprint arXiv:2311.05808*, 2023.
- [27] J. C. Zhao, A. Sharma, A. R. Elkordy, Y. H. Ezzeldin, S. Avestimehr, and S. Bagchi, “Loki: Large-scale data reconstruction attack against federated learning through model manipulation,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2024, pp. 1287–1305.
- [28] M. Luby, “LT codes,” in *Proc. 43rd Annu. IEEE Symp. Found. Comput. Sci. (FOCS)*, 2002, pp. 271–280.
- [29] H. Raynaud, “Sur l’enveloppe convexe des nuages de points aléatoires dans R^n . I,” *J. Appl. Probab.*, vol. 7, no. 1, pp. 35–48, 1970.
- [30] Q. Li, L. Luo, A. Gini, C. Ji, Z. Hu, X. Li, C. Fang, J. Shi, and X. Hu, “Perfect gradient inversion in federated learning: A new paradigm from the hidden subset sum problem,” *arXiv preprint arXiv:2409.14260*, 2024.
- [31] H. Yin, A. Mallya, A. Vahdat, J. M. Álvarez, J. Kautz, and P. Molchanov, “See through gradients: Image batch recovery via GradInversion,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 16332–16341.
- [32] T. Dang, O. Thakkar, S. I. Ramaswamy, R. Mathews, P. Chin, and F. Beaufays, “Revealing and protecting labels in distributed training,” *arXiv preprint arXiv:2111.00556*, 2021.
- [33] K. Ma, Y. Sun, J. Cui, D. Li, Z. Guan, and J. Liu, “Instance-wise batch label restoration via gradients in federated learning,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [34] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009.
- [35] L. Deng, “The MNIST database of handwritten digit images for machine learning research,” *IEEE Signal Process. Mag.*, vol. 29, no. 6, pp. 141–142, 2012.
- [36] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik, “EMNIST: An extension of MNIST to handwritten letters,” *arXiv preprint arXiv:1702.05373*, 2017.
- [37] H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms,” *arXiv preprint arXiv:1708.07747*, 2017.
- [38] I. J. Goodfellow, Y. Bulatov, J. Ibarz, S. Arnaud, and V. Shet, “Multi-digit number recognition from street view imagery using deep convolutional neural networks,” *arXiv preprint arXiv:1312.6082*, 2013.
- [39] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2009, pp. 248–255.
- [40] J. Reyes-Ortiz, D. Anguita, A. Ghio, L. Oneto, and X. Parra, “Human activity recognition using smartphones,” *UCI Machine Learning Repository*, 2013, doi: 10.24432/C54S4K.
- [41] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, Y. W. Teh and M. Titterton, Eds., vol. 9. Chia Laguna Resort, Sardinia, Italy: PMLR, 13–15 May 2010, pp. 249–256.